Docker : Conteneurisation, Images et Orchestration légère

Module : Cloud Computing, Virtualisation & DevOps — Master Sciences et Techniques

Pr. A. ABASSI

Département de Physique Appliquée
Laboratoire LAVETE Lab – FST Settat

Faculté des Sciences et Techniques de Settat
Université Hassan 1er

October 21, 2026

Sommaire

1. Partie VI — Réseau Docker
2. Partie VII — Persistance des données : volumes
3. Partie VIII — Docker Compose
4. Partie IX — Bonnes pratiques
5. Partie X — Docker et le Cloud/DevOps
6. Étude de cas de synthèse
7. Synthèse
8. Mini-quiz
9. Corrigé du mini-exercice Dockerfile
10. TP guidé
11. Conclusion

1. Partie VI — Réseau Docker

Mode réseau `bridge` : Principe

Fonctionnement

Le mode `bridge` connecte les conteneurs à un réseau virtuel Docker. C'est le mode utilisé par défaut.

- Le conteneur possède sa propre adresse IP interne.
- Les conteneurs peuvent communiquer sur le réseau Docker.
- Pour accéder à une application depuis l'hôte, on peut publier un port avec l'option `-p`.

Mode réseau `bridge` : Commande

Exemple

```
docker run -d --name web1 \
--network bridge -p 8080:80 nginx
```

- `-d` : exécuter le conteneur en arrière-plan.
- `-name web1` : donner le nom `web1`.
- `-p 8080:80` : associer le port 80 de l'hôte au port 80 du conteneur.

Mode `bridge` : tester l'accès au serveur Web

Après avoir lancé le conteneur, ouvrez un navigateur sur la machine hôte et saisissez :

```
http://localhost:8080
```

Commandes de vérification

```
docker ps
docker port web1
docker logs web1
```

- `docker ps` : afficher les conteneurs en cours.
- `docker port web1` : afficher les ports publiés.
- `docker logs web1` : consulter les journaux.

Attention

`-p 8080:80` signifie que le port 8080 de l'hôte redirige les connexions vers le port 80 du conteneur. Sans `-p`, le serveur n'est pas publié de cette manière sur l'hôte.

Mode `bridge` : héberger sa propre page Web

Pour remplacer la page par défaut de Nginx par votre propre application (un fichier `index.html` local) :

1. Créer une page simple sur l'hôte

```
echo "<h1>Mon application Docker</h1>" > index.html
```

2. Monter le fichier dans le conteneur

```
docker run -d --name mon-app -v "${PWD}/index.html:/usr/share/nginx/html/index.html:ro" -p
    8080:80 nginx
```

- `-v ...` : associe le fichier local au dossier web de Nginx.
- `:ro` : monte le fichier en lecture seule (*read-only*).

Mode réseau `host`

Principe

Le conteneur partage directement la pile réseau de la machine hôte (pas d'IP séparée, pas de NAT).

```
docker run -d --name web-host --network host nginx
```

- Nginx écoute directement sur le port 80 de l'hôte (`http://localhost:80`).
- L'option `-p` n'est ni nécessaire ni compatible avec ce mode.

Attention sous Windows / macOS

Ce mode est natif sous Linux. Avec Docker Desktop sous Windows/macOS, son comportement diffère (la machine virtuelle intermédiaire gère l'hôte virtuellement).

Mode réseau `none` : Principe

Principe

Le mode réseau `none` isole le conteneur des réseaux Docker. Il ne dispose donc pas de connectivité réseau externe.

Conséquences

- Le conteneur n'est connecté à aucun réseau Docker.
- Il ne peut normalement pas communiquer avec Internet ni avec les autres conteneurs par le réseau.
- L'interface locale `lo` reste normalement disponible pour les communications internes.
- Le conteneur peut toujours exécuter des programmes et traiter des fichiers locaux.

Exemple d'utilisation

Exécuter un programme Python qui effectue des calculs locaux sans avoir besoin d'accéder à Internet.

Mode `none` : Décomposition de la commande

Commande complète

```
docker run --rm --network none python:3.11-slim python -c \
"import socket; print(socket.if_nameindex())"
```

Rôle de chaque élément

`docker run` Crée et démarre un conteneur.

`-rm` Supprime automatiquement le conteneur lorsqu'il s'arrête.

`-network none` Ne connecte le conteneur à aucun réseau Docker.

`python:3.11-slim` Image contenant une version légère de Python 3.11.

`python -c` Exécute directement du code Python fourni sous forme de texte.

Remarque

Le caractère \ permet de poursuivre la commande sur la ligne suivante dans un terminal Linux ou Bash. Il peut être omis si la commande tient sur une ligne.

Mode `none` : Comprendre le code Python

Code exécuté

```
import socket
print(socket.if_nameindex())
```

- **`socket.if_nameindex()`** : retourne les interfaces réseau visibles par le conteneur.

Résultat obtenu en mode `none`

```
[(1, 'lo')]
```

L'unique interface `lo` (*loopback*) confirme l'absence totale de carte réseau externe (`eth0`).

À retenir

Isolation maximale : aucun échange réseau externe n'est possible dans ce mode.

Mode `none` : Comparaison pratique

Test 1 : Mode `none`

```
docker run --rm --network none python:3.11-slim
    python -c "import socket; print(socket.
    if_nameindex())"
```

Résultat : `[(1, 'lo')]` (isolé)

Test 2 : Réseau défaut

```
docker run --rm python:3.11-slim python -c "import
    socket; print(socket.if_nameindex())"
```

Résultat : `[(1, 'lo'), (2, 'eth0')]`

- `lo` : Boucle locale (interne). **`eth0`** : Interface réseau active.

Conclusion

Le mode `none` supprime toute interface externe (`eth0`), garantissant une isolation réseau totale, contrairement au réseau par défaut.

Réseau personnalisé : créer et connecter

Principe

Un réseau bridge personnalisé permet aux conteneurs connectés de communiquer par leur nom grâce au DNS intégré de Docker.

Listing 1: Étape 1 : Créer le réseau

```
docker network create mon-reseau
```

Listing 2: Étape 2 : Lancer les conteneurs connectés

```
docker run -d --name web2 --network mon-reseau nginx
docker run -d --name client --network mon-reseau python:3.11-slim python -m http.server 8000
```

- **web2** : exécute le serveur Nginx.
- **client** : exécute un serveur Python sur le port 8000.
- Les deux conteneurs s'isolent ensemble sur `mon-reseau`.

Réseau personnalisé : tester la communication

Tester depuis le conteneur `client`

```
docker exec client python -c "import urllib.request; print(urllib.request.urlopen('http://
    web2').status)"
```

- **`docker exec`** : exécute la commande dans le conteneur `client`.
- **`http://web2`** : résolution automatique par nom d'hôte Docker.
- **Résultat attendu** : 200 (succès de la requête HTTP).

Listing 3: Inspection du réseau

```
docker network ls && docker network inspect mon-reseau
```

Important

Pas de publication de port nécessaire sur l'hôte : la communication s'établit directement en interne entre les conteneurs du même réseau.

Comparaison des modes réseau Docker

Mode	Commande	Utilisation principale
`bridge`	`docker run -network bridge ...`	Réseau virtuel ; publier un port avec `-p` pour accéder au service depuis l'hôte.
`host`	`docker run -network host ...`	Partage de la pile réseau de l'hôte ; surtout sous Linux.
`none`	`docker run -network none ...`	Conteneur sans connectivité réseau externe.
Personnalisé	`docker network create mon-reseau`	Communication entre conteneurs par leur nom, sur un réseau partagé.

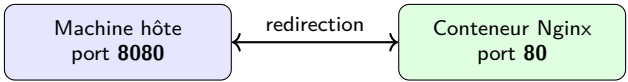
Nettoyer les conteneurs et le réseau de démonstration

```
docker rm -f web1 web-host web2 client
docker network rm mon-reseau
```

À retenir : l'option `-network` choisit le réseau du conteneur ; l'option `-p` publie un port vers l'hôte lorsqu'elle est applicable.

La redirection de ports -p

```
docker run -d -p 8080:80 nginx
```



`-p HOTE:CONTENEUR` : tout trafic arrivant sur le port **8080 de l'hôte** est redirigé vers le port **80 à l'intérieur du conteneur**, où Nginx écoute réellement.

Question de vérification

Avec `-p 3000:80`, sur quel port faut-il ouvrir le navigateur pour accéder à l'application ? Le port 80 est-il encore utilisable pour un autre service sur l'hôte ?

Réseaux personnalisés et communication entre conteneurs

```
docker network create monreseau
docker run -d --name db --network monreseau postgres
docker run -d --name api --network monreseau monapp:1.0
```

- `docker network create` : crée un réseau `bridge` personnalisé.
- Deux conteneurs sur le **même réseau** personnalisé peuvent se joindre **par leur nom** (ici, `api` peut contacter `db` directement, sans connaître son IP).

À retenir

Docker fournit une résolution DNS interne automatique entre conteneurs d'un même réseau personnalisé — le nom du conteneur fait office de nom d'hôte.

Communication API ↔ PostgreSQL sur un réseau personnalisé

Chaîne de connexion de l'API (ex: .NET / FastAPI)

L'API utilise directement le nom du conteneur `db` comme nom d'hôte :

```
# Exemple pour FastAPI / Python :
DATABASE_URL = "postgresql://user:pass@db:5432/mabase"

# Exemple pour .NET Core :
"Host=db;Port=5432;Database=mabase;Username=user;Password=pass"
```

- **Pas d'adresse IP en dur** : Grâce au DNS interne de Docker, `db` est résolu automatiquement.
- **Isolation** : Les conteneurs communiquent en toute sécurité sur `monreseau` sans exposer inutilement PostgreSQL sur l'hôte.

PostgreSQL : Lancement et création de la base

Pour lancer PostgreSQL avec des variables d'environnement et créer une table :

1. Lancer le conteneur avec variables d'environnement

```
docker run -d --name db --network monreseau \
-e POSTGRES_USER=admin \
-e POSTGRES_PASSWORD=secret \
-e POSTGRES_DB=produits_db \
postgres:15
```

2. Créer une table de test via le client psql

```
docker exec -it db psql -U admin -d produits_db -c \
"CREATE TABLE produits (id SERIAL PRIMARY KEY, nom VARCHAR(100));"

docker exec -it db psql -U admin -d produits_db -c \
"INSERT INTO produits (nom) VALUES ('Laptop'), ('Souris');"
```

Développement d'une API simple (FastAPI)

Voici un exemple minimal d'API en Python (‘main.py‘) qui se connecte à PostgreSQL (‘db‘) :

Code source (main.py)

```
from fastapi import FastAPI
import psycopg2

app = FastAPI()

@app.get("/produits")
def get_produits():
conn = psycopg2.connect(
host="db", database="produits_db",
user="admin", password="secret"
)
cursor = conn.cursor()
cursor.execute("SELECT id, nom FROM produits;")
resultats = cursor.fetchall()
cursor.close()
conn.close()
return [{"id": r[0], "nom": r[1]} for r in resultats]
```

Conteneurisation de l'API : Le `Dockerfile`

Pour empaqueter l'application Python dans une image Docker :

Contenu du fichier `Dockerfile`

```
# Utiliser une image officielle Python
FROM python:3.10-slim

# Définir le dossier de travail
WORKDIR /app

# Installer les dépendances
RUN pip install fastapi uvicorn psycopg2-binary

# Copier le code source
COPY main.py .

# Exposer le port de l'API et lancer uvicorn
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- `psycopg2-binary` : Pilote Python pour communiquer avec PostgreSQL.
- `uvicorn` : Serveur ASGI pour exécuter l'application FastAPI.

Build et test de l'architecture complète

1. Construire l'image de l'API

```
docker build -t monapp:1.0 .
```

2. Lancer l'API sur le réseau personnalisé

```
docker run -d --name api --network monreseau \
-p 8000:8000 monapp:1.0
```

3. Tester depuis la machine hôte

Ouvrez votre navigateur ou utilisez `curl` :

```
curl http://localhost:8000/produits
# Résultat attendu : [{"id": 1, "nom": "Laptop"}, ...]
```

2. Partie VII — Persistance des données : volumes

Le problème : les conteneurs sont éphémères

Par défaut

Toutes les données écrites **à l'intérieur** d'un conteneur sont perdues dès que ce conteneur est **supprimé** (`docker rm`) — la couche modifiable du conteneur disparaît avec lui.

Pour conserver des données de façon durable (base de données, fichiers uploadés...), il faut les stocker **en dehors** du système de fichiers du conteneur : c'est le rôle des **volumes**.

Pourquoi conserver les données d'un conteneur ?

Exemple : application de gestion des étudiants

Enregistrement dans une base de données interne : nom, numéro d'inscription, notes, résultats.

Le problème (Cycle de vie éphémère)

Le système de fichiers d'un conteneur est supprimé avec lui. Les données y étant stockées sont définitivement perdues lors de la suppression.

La solution

Utiliser un stockage persistant, indépendant du cycle de vie du conteneur :

- **Volumes Docker** (recommandé et géré par Docker).
- **Bind mounts** (lier un dossier de la machine hôte).

Le principe du montage dans Docker

Définition

Rendre un espace de stockage accessible à l'intérieur d'un conteneur à un emplacement précis.

Machine Hôte
Stockage / Données persistantes

↓ **Montage**

Conteneur Docker
Dossier interne (ex: `/app/data`)

- **Transparence** : Accès comme des fichiers ordinaires.
- **Persistance** : Survie des données malgré la suppression du conteneur.

Le volume Docker : définition

Définition

Espace de stockage géré par Docker et totalement indépendant du cycle de vie du conteneur.

Emplacement physique

Géré par l'hôte. Sous Linux :

`/var/lib/docker/volumes/`

Invisible ou abstrait pour l'utilisateur sous Windows / Mac (Docker Desktop).

Exemple (PostgreSQL)

Base de données stockant ses fichiers dans :

`/var/lib/postgresql/data`

Relié au volume nommé `data_db` pour garantir la persistance.

À retenir

Le chemin interne appartient au conteneur, mais les données persistent sur l'hôte au-delà de sa suppression.

Volume Docker : création et utilisation

Étape 1 : Créer le volume

```
docker volume create data_db
```

Étape 2 : Lancer PostgreSQL avec ce volume

```
docker run -d --name ma_bdd -v data_db:/var/lib/postgresql/data -e POSTGRES_PASSWORD=
    Etudiant123 postgres
```

- `-v data_db:...` : Monte le volume dans le dossier interne de persistance de PostgreSQL.
- `-e POSTGRES_PASSWORD=...` : Définit le mot de passe initial de l'administrateur.

Sécurité (TP)

Ce mot de passe simple est purement pédagogique et ne doit jamais être utilisé en production.

Volume Docker : vérifier la persistance

1. Inspection du volume

```
docker volume ls
docker volume inspect data_db
```

2. Suppression du conteneur

```
docker rm -f ma_bdd
```

`-f` force l'arrêt et la suppression.

3. Relance avec le même volume

```
docker run -d --name ma_bdd2 -v data_db:/var/lib/
      postgresql/data -e POSTGRES_PASSWORD=Etudiant123
       postgres
```

Conclusion

Le volume `data_db` survit à la suppression de `ma_bdd`. Le nouveau conteneur `ma_bdd2` récupère instantanément toutes les données précédentes.

Exemple pratique : lancer PostgreSQL avec Docker I

Objectif

Déployer un serveur PostgreSQL dans un conteneur Docker et conserver les données grâce à un volume persistant.

Commande Docker

```
docker run -d --name ma_bdd \
-v data_db:/var/lib/postgresql \
-e POSTGRES_PASSWORD=Etudiant123 \
-p 5432:5432 postgres
```

- `-d` : exécution en arrière-plan.
- `-name ma_bdd` : nom du conteneur.

Exemple pratique : lancer PostgreSQL avec Docker II

- `-v data_db:/var/lib/postgresql` : stockage persistant.
- `-e POSTGRES_PASSWORD` : mot de passe de l'utilisateur `postgres`.
- `-p 5432:5432` : publication du port PostgreSQL.
- `postgres` : image officielle utilisée.

Vérifier le démarrage du conteneur

1. Afficher les conteneurs en cours / tous

```
docker ps
docker ps -a
```

2. Consulter les journaux du serveur

```
docker logs ma_bdd
```

À retenir

La création d'un conteneur ne garantit pas que le service fonctionne correctement. La colonne `STATUS` et les journaux permettent de vérifier son état.

PostgreSQL : corriger le montage du volume I

Problème rencontré

Les images officielles PostgreSQL récentes utilisent une organisation des données spécifique.
Le montage sur `/var/lib/postgresql/data` peut provoquer une erreur au démarrage.

Ancienne commande : montage inadapté

```
docker run -d --name ma_bdd \
-v data_db:/var/lib/postgresql/data \
-e POSTGRES_PASSWORD=Etudiant123 \
-p 5432:5432 postgres
```

PostgreSQL : corriger le montage du volume II

Commande corrigée

```
docker run -d --name ma_bdd \
-v data_db:/var/lib/postgresql \
-e POSTGRES_PASSWORD=Etudiant123 \
-p 5432:5432 postgres
```

Recréer le conteneur après une erreur I

1. Supprimer le conteneur arrêté :

```
docker rm ma_bdd
```

2. Recréer avec le bon montage :

```
docker run -d --name ma_bdd \
-v data_db:/var/lib/postgresql \
-e POSTGRES_PASSWORD=Etudiant123 \
-p 5432:5432 postgres
```

3. Vérifier le démarrage :

```
docker ps
docker logs ma_bdd
```

Recréer le conteneur après une erreur II

Attention

`docker rm` supprime le conteneur, mais conserve normalement le volume nommé. Ne supprimez pas le volume si vous souhaitez préserver les données.

Créer une base de données PostgreSQL I

Étape 1 : ouvrir le terminal PostgreSQL

```
docker exec -it ma_bdd psql -U postgres
```

Étapes 2, 3 et 4 : Base et Table

```
CREATE DATABASE etudiants_db;
\c etudiants_db
CREATE TABLE etudiants (
id SERIAL PRIMARY KEY,
nom VARCHAR(100),
age INTEGER
);
```

Insérer et consulter des données

Insérer des étudiants

```
INSERT INTO etudiants (nom, age)
VALUES ('Ahmed', 21), ('Sara', 22), ('Youssef', 20);
```

Commandes utiles

```
SELECT * FROM etudiants; -- Afficher les données
\l                       -- Afficher les bases
\q                       -- Quitter PostgreSQL
```

Vérifier la persistance des données I

Gestion des volumes

```
docker volume ls
docker volume inspect data_db
```

Supprimer et recréer

```
docker rm -f ma_bdd
docker run -d --name ma_bdd \
-v data_db:/var/lib/postgresql \
-e POSTGRES_PASSWORD=Etudiant123 \
-p 5432:5432 postgres
```

Vérifier la persistance des données II

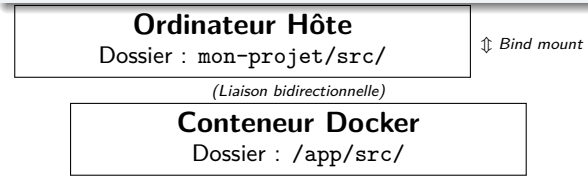
Conclusion

Le volume permet de conserver les données indépendamment du cycle de vie du conteneur.

Bind mount : le principe

Définition

Un **bind mount** relie directement un dossier ou fichier de l'hôte à un emplacement dans un conteneur.



Exemple concret

Modification de `app.py` sur l'ordinateur de l'enseignant → instantanément visible dans le conteneur, sans reconstruction d'image.

Pourquoi utiliser un bind mount ?

Lors du développement d'une application Flask, le programmeur modifie régulièrement son code source.

Sans bind mount

Après chaque modification du code, il peut être nécessaire de reconstruire l'image Docker pour intégrer les changements.

Avec bind mount

Les fichiers du projet présents sur l'ordinateur sont directement accessibles depuis le conteneur Docker.

Avantage principal

Modifier le code Python sur l'ordinateur sans reconstruire l'image Docker à chaque changement.

Principe de fonctionnement d'un bind mount

Un bind mount associe un dossier de l'ordinateur hôte à un emplacement précis dans le conteneur.



- L'hôte contient les fichiers originaux du projet.
- Le conteneur accède à ces fichiers dans le dossier `/app`.
- Les modifications réalisées sur l'hôte sont visibles dans le conteneur.

Organisation du projet Flask

Avant de créer le conteneur, préparons les fichiers de notre application.

Structure du projet

```
mon-projet-flask/
|-- app.py
|-- requirements.txt
‘-- Dockerfile
```

`app.py` Contient le code de l'application Flask.

`requirements.txt` Liste les bibliothèques Python nécessaires.

`Dockerfile` Décrit comment construire l'image Docker.

Attention

Le fichier doit s'appeler exactement `Dockerfile`, sans extension `.txt`.

Exemple : créer une application Flask

Le fichier `app.py` contient le code suivant :

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def accueil():
    return "Bonjour depuis Flask avec Docker !"

if __name__ == "__main__":
    app.run(host="0.0.0.0",
    port=5000,
    debug=True)
```

- `Flask(__name__)` crée l'application web.
- `@app.route("/")` définit la page d'accueil.
- `host="0.0.0.0"` permet d'accepter les connexions provenant de l'extérieur du conteneur.
- `port=5000` définit le port d'écoute.
- `debug=True` active le mode de développement et le rechargement automatique du code.

Dockerfile : préparer l'environnement Flask

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 5000

CMD ["python", "app.py"]
```

`FROM`	Utilise une image de base contenant Python.
`WORKDIR`	Définit le dossier de travail /app.
`COPY`	Copie le fichier des dépendances.
`RUN`	Installe Flask pendant la construction de l'image.
`EXPOSE`	Documente le port utilisé par l'application.
`CMD`	Définit la commande exécutée au démarrage.

Pourquoi ne pas copier app.py dans l'image ?

Dans un projet classique, on pourrait ajouter au Dockerfile :

Instruction habituelle

```
COPY app.py .
```

Cette instruction copie le code source dans l'image au moment de sa construction.

Avec un bind mount

Le code est fourni depuis l'hôte au démarrage du conteneur. Il n'est donc pas nécessaire de le copier dans l'image pour cette démonstration.

- L'image contient Python et les dépendances installées.
- Le dossier de l'hôte fournit le code source.
- Le montage masque les fichiers déjà présents dans le dossier cible du conteneur.

Étape 1 : construire l'image Docker

Ouvrir un terminal dans le dossier contenant le Dockerfile, puis exécuter :

Commande de construction

```
docker build -t flask-dev:1.0 .
```

`docker build` Construit une image Docker.

`-t` Attribue un nom et un tag à l'image.

`flask-dev:1.0` Nom et version de l'image.

`.` Utilise le dossier courant comme contexte de construction.

Résultat

Docker installe les dépendances et prépare une image appelée `flask-dev`, version `1.0`.

Étape 3 : lancer Flask avec un bind mount

Depuis le dossier du projet, exécuter sous PowerShell ou Linux :

```
docker run -d \
--name flask-dev \
-p 5000:5000 \
--mount "type=bind,source=.,target=/app" \
flask-dev:1.0
```

`-d` Lance le conteneur en arrière-plan.

`-name flask-dev` Donne un nom au conteneur.

`-p 5000:5000` Publie le port 5000 de l'hôte vers le port 5000 du conteneur.

`type=bind` Demande un montage de type bind mount.

`source=.` Désigne le dossier courant de l'hôte.

`target=/app` Monte le dossier dans /app.

Commande complète sur une seule ligne

```
docker run -d --name flask-dev -p 5000:5000 --mount "type=bind,source=.,target=/app" flask-dev:1.0
```

- Le dossier courant doit contenir `app.py`.
- Le port 5000 de l'hôte doit être disponible.
- Docker Desktop doit être démarré sous Windows.

Étape 5 : modifier le code sans reconstruire l'image

Modifier la fonction `accueil()` dans le fichier `app.py` sur l'ordinateur :

```
@app.route("/")
def accueil():
return "Le bind mount fonctionne !"
```

1. Enregistrer le fichier.
2. Flask détecte normalement le changement grâce à `debug=True`.
3. L'application redémarre automatiquement.
4. Actualiser la page dans le navigateur.

Résultat

Le nouveau message s'affiche sans exécuter `docker build` une nouvelle fois.

Synthèse : développer Flask avec Docker

1. Créer `app.py`, `requirements.txt` et `Dockerfile`.
2. Construire l'image avec `docker build -t flask-dev:1.0 .`.
3. Démarrer le conteneur avec un bind mount et la publication du port 5000.
4. Accéder à l'application via `http://localhost:5000`.
5. Modifier le code sur l'hôte et vérifier le résultat.

Idée essentielle

Le bind mount partage les fichiers entre l'hôte et le conteneur. Il facilite le développement, mais ne remplace ni la construction de l'image ni l'installation des dépendances.

En production

Désactiver le mode debug et utiliser un serveur adapté à la production, par exemple Gunicorn.

Volumes Docker vs Bind mounts

Critère	Volume Docker	Bind mount
Gestion	Géré entièrement par Docker	Basé sur un chemin de l'hôte
Emplacement	Zone dédiée gérée par Docker	Fichier ou dossier spécifique de l'hôte
Usage type	Données persistantes (Bases de données)	Code source, configuration de dev
Développement	Moins direct pour éditer le code	Idéal pour éditer directement les fichiers
Portabilité	Indépendant de l'arborescence hôte	Dépend de la structure de l'hôte

Règle pratique

- **Données persistantes / DB** → Volume Docker.
- **Code source / Édition en direct** → Bind mount.

Mini-exercice — Volumes

Un conteneur MySQL est démarré **sans** option `-v`. L'administrateur exécute ensuite `docker rm` dessus par erreur avant d'avoir sauvegardé la base.

1. Les données de la base de données sont-elles récupérables ?
2. Quelle commande aurait dû être utilisée au moment du `docker run` pour éviter ce problème ?

3. Partie VIII — Docker Compose

Le problème : une application, plusieurs conteneurs

Une application réelle combine souvent plusieurs services : une API, une base de données, un cache, un serveur web...

Sans Compose

Il faudrait taper une longue commande `docker run` (avec réseaux, volumes, variables d'environnement) pour **chaque** conteneur, dans le bon ordre, à chaque démarrage. Long, source d'erreurs, non versionnable facilement.

Docker Compose

Un outil qui décrit, dans un **fichier YAML unique** (`docker-compose.yml`), l'ensemble des services d'une application multi-conteneurs, et les démarre/arrête tous ensemble.

C'est le pendant, pour plusieurs conteneurs liés, de ce que le Dockerfile est pour une seule image : de l'**Infrastructure as Code**.

Exemple de `docker-compose.yml`

```
version: "3.9"

services:
  api:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:15
    volumes:
      - data_db:/var/lib/postgresql/data
    environment:
      - POSTGRES_PASSWORD=secret

volumes:
  data_db:
```

Le fichier Compose, ligne par ligne

- `services:` : liste des conteneurs à créer, chacun avec un nom logique (`api`, `db`).
- `build: .` : construit l'image du service `api` à partir du Dockerfile local (au lieu de `image:` pour une image existante).
- `image: postgres:15` : utilise directement une image publique pour le service `db`.
- `ports:` : équivalent de l'option `-p` de `docker run`.
- `environment:` : équivalent de l'option `-e`.
- `depends_on:` : indique l'**ordre de démarrage** — ici, `db` démarre avant `api` (mais n'attend pas que la base soit **prête**, seulement démarrée).
- `volumes:` (en bas) : déclare le volume nommé `data_db` utilisé par le service `db`.

Tous les services d'un même fichier Compose sont automatiquement placés sur un **réseau commun** : `api` peut joindre `db` simplement par son nom, comme vu en Partie VI.

Commandes Docker Compose

```
docker compose up -d
```

Construit (si nécessaire) et démarre **tous les services** définis dans le fichier, en arrière-plan.

```
docker compose ps
docker compose logs -f api
```

Liste les services actifs ; suit les journaux d'un service précis.

```
docker compose down
```

Arrête et supprime tous les conteneurs **et** le réseau créés par Compose (les volumes nommés sont conservés, sauf avec `-v`).

Attention

`docker compose down -v` supprime **aussi** les volumes → perte définitive des données si elles n'ont pas été sauvegardées ailleurs.

4. Partie IX — Bonnes pratiques

Réduire la taille des images

- Préférer les images de base **allégées** : `python:3.11-slim` ou `-alpine` plutôt qu'une image complète.
- Regrouper les commandes `RUN` liées (chaque `RUN` crée une couche) pour limiter le nombre de couches et nettoyer les fichiers temporaires dans la **même** instruction.
- Toujours utiliser un `.dockerignore` (voir Partie IV).

Exemple : regrouper les RUN

```
RUN apt-get update && apt-get install -y curl \
    && rm -rf /var/lib/apt/lists/*
```

Le *multi-stage build*

Permet de construire l'application dans une **étape intermédiaire** (avec tous les outils de compilation), puis de ne copier **que le résultat** dans l'image finale, beaucoup plus légère.

```
# Étape 1 : compilation
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

# Étape 2 : image finale, allégée
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

À retenir

L'image finale ne contient **ni** Node.js **ni** les outils de build — seulement les fichiers statiques générés — ce qui réduit drastiquement sa taille et sa surface d'attaque.

Sécurité : ne pas exécuter en root

Par défaut

Un processus dans un conteneur Docker s'exécute, par défaut, en tant qu'utilisateur `root` **à l'intérieur** du conteneur — risque en cas de faille permettant de sortir du conteneur.

```
RUN adduser --disabled-password appuser
USER appuser
```

Crée un utilisateur non privilégié et bascule dessus pour l'exécution — application directe du principe du **moindre privilège** déjà vu pour les utilisateurs Linux (Chapitre 2).

Autres bonnes pratiques de sécurité

- Ne jamais intégrer de secrets en dur dans le Dockerfile
- Utiliser des images officielles ou vérifiées, avec des tags de version précis
- Scanner régulièrement les images (`docker scan` ou outils tiers) pour détecter des vulnérabilités connues

5. Partie X — Docker et le Cloud/DevOps

Registres d'images dans le Cloud

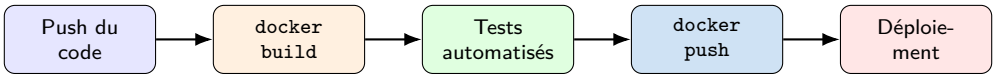
Docker Hub est le registre public par défaut, mais chaque grand Cloud propose son propre registre **privé** :

Fournisseur	**Registre**
Docker (public)	Docker Hub
AWS	Amazon ECR (Elastic Container Registry)
Azure	Azure Container Registry (ACR)
Google Cloud	Google Container Registry / Artifact Registry

Pourquoi un registre privé ?

Héberger ses propres images sans les rendre publiques, avec un contrôle d'accès via IAM (voir Chapitre 2) et une intégration directe avec les services de déploiement du Cloud.

Docker dans un pipeline CI/CD



- Chaque modification du code déclenche automatiquement la construction d'une nouvelle image.
- L'image est testée, puis publiée (*push*) vers un registre.
- Le service de déploiement (Cloud, Kubernetes...) récupère la nouvelle image et met à jour l'application en cours d'exécution.

À retenir

Docker rend le déploiement **reproductible** : l'image testée en intégration continue est **exactement** celle exécutée en production.

Docker et Kubernetes : quelle relation ?

- Docker permet de **construire** et **exécuter** un conteneur.
- Quand une application compte des dizaines/centaines de conteneurs répartis sur plusieurs machines, il faut les **orchestrer** : les répartir, les redémarrer en cas de panne, les mettre à l'échelle automatiquement.
- C'est le rôle de **Kubernetes** (ou de solutions plus simples comme Docker Compose/Swarm, pour de petites échelles).

À retenir

Docker construit et fait tourner **un** conteneur ; Kubernetes orchestre **une flotte** de conteneurs à grande échelle (sujet détaillé dans un chapitre dédié).

6. Étude de cas de synthèse

Scénario intégrateur

Contexte

Une petite équipe développe une API Flask qui doit se connecter à une base PostgreSQL. Elle souhaite un environnement de développement **reproductible**, démarrable en une seule commande, avec les données de la base conservées entre deux redémarrages.

À déterminer :

1. le Dockerfile de l'API ;
2. le fichier `docker-compose.yml` complet (API + base) ;
3. la gestion de la persistance des données ;
4. les commandes de démarrage et de vérification.

Solution (1/2) — Dockerfile de l'API

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Image légère (*slim*), dépendances installées avant le code applicatif pour profiter du cache de build.

Solution (2/2) — Docker Compose et vérification

```
services:
  api:
    build: .
    ports: ["5000:5000"]
    depends_on: [db]
  db:
    image: postgres:15
    volumes: ["data_db:/var/lib/postgresql/data"]
    environment: ["POSTGRES_PASSWORD=secret"]
volumes:
  data_db:
```

```
docker compose up -d
docker compose ps
docker compose logs -f api
```

Le volume nommé `data_db` garantit que les données PostgreSQL survivent à un `docker compose down` (sans `-v`).

7. Synthèse

Tableau de synthèse final

Concept	Savoir-faire
Conteneur vs VM	Expliquer la différence et les mécanismes Linux sous-jacents
Architecture Docker	Situer CLI, daemon, registry
Image vs Conteneur	Distinguer modèle et instance
Commandes de base	`run`, `ps`, `exec`, `logs`, `images`
Dockerfile	Écrire et optimiser (cache, multi-stage)
Réseau	Choisir un mode, rediriger un port
Volumes	Assurer la persistance des données
Docker Compose	Décrire une application multi-conteneurs
Sécurité	Appliquer le moindre privilège dans un conteneur
Cloud/DevOps	Situer Docker dans un pipeline CI/CD et face à Kubernetes

8. Mini-quiz

Mini-quiz (1/2)

1. Un conteneur embarque-t-il son propre noyau OS ?
2. Quelle est la différence entre une image et un conteneur ?
3. Quel composant Docker stocke les images publiées ?
4. Que fait `docker run -d -p 8080:80 nginx` ?
5. Pourquoi placer `COPY requirements.txt .` avant `COPY . .` ?

Mini-quiz (2/2)

6. Quelle est la différence entre `CMD` et `ENTRYPOINT` ?
7. Pourquoi utiliser un volume plutôt que d'écrire dans le système de fichiers du conteneur ?
8. À quoi sert `depends_on` dans un fichier Compose ?
9. Quel est l'intérêt d'un multi-stage build ?
10. Docker remplace-t-il Kubernetes ?

Corrigé du mini-quiz

1. Non : il partage le noyau de la machine hôte.
2. L'image est un modèle en lecture seule ; le conteneur est une instance en cours d'exécution de cette image.
3. Le **registry** (ex. Docker Hub).
4. Démarre un conteneur Nginx en arrière-plan, port 80 du conteneur accessible via le port 8080 de l'hôte.
5. Pour profiter du cache de build : si seul le code change (pas les dépendances), l'étape d'installation n'est pas refaite.
6. `CMD` est remplaçable en ligne de commande ; `ENTRYPOINT` reste fixe (les arguments s'y ajoutent).
7. Un conteneur est éphémère : ses données sont perdues à sa suppression, sauf si stockées dans un volume externe.
8. Il définit l'ordre de démarrage des services (mais n'attend pas qu'un service soit *prêt*, seulement démarré).
9. Réduire la taille de l'image finale en n'y copiant que les artefacts compilés, sans les outils de build.
10. Non : Docker construit/exécute des conteneurs ; Kubernetes les orchestre à grande échelle. Rôles complémentaires.

9. Corrigé du mini-exercice Dockerfile

Corrigé — Dockerfile Node.js à corriger

Problèmes identifiés

1. `FROM node` utilise l'image complète (lourde) et sans version fixée (tag `latest` implicite) : préférer `node:20-slim` par exemple.
2. `COPY . .` est placé **avant** `RUN npm install` : tout changement de code (même sans toucher aux dépendances) invalide le cache et force une réinstallation complète des paquets.

Version corrigée

```
FROM node:20-slim
WORKDIR /app
COPY package*.json .
RUN npm install
COPY . .
CMD ["node", "server.js"]
```

10. TP guidé

TP guidé (45–60 min)

1. Vérifier l'installation : `docker -version`, `docker run hello-world`
2. Démarrer un conteneur Nginx et y accéder via le navigateur (`-p`)
3. Explorer le conteneur avec `docker exec -it ... bash`
4. Écrire un Dockerfile pour une petite application (Python ou Node.js)
5. Construire l'image (`docker build`) et la démarrer (`docker run`)
6. Ajouter un volume pour la persistance des données
7. Écrire un `docker-compose.yml` combinant l'application et une base de données
8. Démarrer l'ensemble avec `docker compose up -d` et vérifier les journaux
9. Arrêter proprement avec `docker compose down`

Attention

`docker rm -f`, `docker rmi` et `docker compose down -v` sont irréversibles : vérifier qu'aucune donnée importante n'est perdue avant de les exécuter.

11. Conclusion

Résumé du chapitre

- Un conteneur **partage le noyau** de l'hôte : léger, rapide, portable — à ne pas confondre avec une VM.
- L'architecture Docker repose sur trois éléments : **CLI**, **daemon**, **registry**.
- Une **image** est un modèle ; un **conteneur** en est une instance exécutée.
- Le **Dockerfile** décrit, en couches, comment construire une image reproductible.
- **Réseau** (`-p`) et **volumes** (`-v`) permettent de communiquer avec l'extérieur et de persister les données.
- **Docker Compose** orchestre plusieurs conteneurs liés comme une seule application.
- Bonnes pratiques : images légères, multi-stage builds, moindre privilège, registres privés dans le Cloud.

Prochaine étape : orchestrer à grande échelle avec Kubernetes !